

IMPLANTACIÓN DE APLICACIONES WEB

Arquitectura y tecnología de las aplicaciones web

ÍNDICE

/ 1. Introducción y contextualización práctica	4
/ 2. Aplicaciones web y aplicaciones de escritorio	5
2.1. Aplicaciones de escritorio	5
2.2. Aplicaciones web	5
2.3. Diferencias entre aplicaciones web y aplicaciones de escritorio	5
/ 3. Arquitectura cliente-servidor	6
3.1. Proceso de comunicación	6
3.2. Características	6
3.3. Ventajas e inconvenientes	6
/ 4. Protocolos de comunicación	7
4.1. Protocolo HTTP	7
4.2. Protocolo HTTPS	7
/ 5. Caso práctico 1: “Selección de arquitecturas”	8
/ 6. Tecnologías en el lado del cliente	8
6.1. Sistema operativo cliente	8
6.2. Navegador web	9
/ 7. Tecnologías en el lado del servidor	9
7.1. Sistema operativo servidor	9
7.2. Servidor Web	10
7.3. Servidor de bases de datos	10
7.4. Traductor del lenguaje (lógica del programa)	11

ÍNDICE

/ 8. Infraestructura de tecnologías de servidor XAMP	11
/ 9. Conexión con el servidor remoto	12
9.1. Protocolo FTP	12
9.2. FTP Seguro: FTPS	12
9.3. SFTP	12
9.4. Herramientas de transferencia	12
/ 10. Caso práctico 2: “Instalamos XAMP”	13
/ 11. Resumen y resolución del caso práctico de la unidad	13
/ 12. Bibliografía	14

OBJETIVOS

Comprender el concepto de aplicación web.

Conocer las principales arquitecturas y tecnologías relacionadas con las aplicaciones web.

Identificar el software necesario para el correcto funcionamiento de las aplicaciones web.

/ 1. Introducción y contextualización práctica

En esta unidad estudiaremos el concepto de aplicación web, analizaremos en detalle la arquitectura cliente-servidor e identificaremos las diferentes tecnologías necesarias para dar soporte a las aplicaciones web.

Una aplicación web no necesita grandes recursos por parte del usuario, que hará uso de ellas a través de un *software* de bajos recursos conocido como **cliente**. La ejecución real de la aplicación se hace en otro dispositivo conocido como **servidor**, y en el lado del cliente simplemente se obtienen los resultados.

En la actualidad nos encontramos con un enjambre de multitud de equipos interconectados entre sí, funcionando como servidores y como clientes, haciendo uso de redes como **Internet**.

A continuación, vamos a plantear un caso práctico a través del cual podremos aproximarnos de forma práctica a la teoría de este tema.

Escucha el siguiente audio donde planteamos la contextualización práctica de este tema, encontrarás su resolución en el apartado ‘Resumen y resolución del caso práctico’.



Fig.1. Ejemplo de Arquitectura Cliente/Servidor en la actualidad.



Audio intro. "Diseñamos la arquitectura de nuestra aplicación web"

<http://bit.ly/3Y8kf0v>





/ 2. Aplicaciones web y aplicaciones de escritorio

Habitualmente, las corporaciones facilitan versiones de escritorio y versiones web de sus aplicaciones para que se pueda hacer uso de ambas y aprovechar las ventajas de cada una en función de las necesidades de cada usuario.

2.1. Aplicaciones de escritorio

Las aplicaciones de escritorio **se ejecutan íntegramente en el dispositivo del usuario**, almacenando la información esencial y haciendo uso de los recursos íntegros del dispositivo cliente. Es posible que establezcan conexiones a través de una red para tener acceso a cierta información, pero no para la ejecución de su lógica.

Ejemplos: Word, Excel, Notas, Thunderbird, Outlook, Calendario, Calculadora, WhatsApp, etc.

2.2. Aplicaciones web

Las aplicaciones web hacen uso de unos recursos mínimos en el lado del cliente, descargando el grueso de la **ejecución** de la lógica **en el lado del servidor**. De esta forma, el cliente no necesita disponer de grandes recursos, ni de almacenamiento de los datos.

Es preciso que para el correcto funcionamiento de la aplicación se establezca una conexión entre el cliente y el servidor para que el usuario realice las peticiones y el servidor envíe los resultados. Habitualmente, la red que se utiliza es Internet.

Ejemplos: Google Docs, Facebook, WhatsApp Web, Microsoft Office 365, GMail, etc.



Investigamos...

Investiga ejemplos de aplicaciones que tengan versiones de escritorio y versiones web. Por ejemplo, WhatsApp, que puedes usar tanto en su versión de escritorio como en su versión web a través de un navegador.

2.3. Diferencias entre aplicaciones web y aplicaciones de escritorio

En la tabla 1 podemos encontrar las diferencias entre ambas.

	APLICACIONES WEB	APLICACIONES DE ESCRITORIO
Recursos	No requiere instalar software	Requiere la instalación de la aplicación
Velocidad	Las conexiones entre cliente y servidor ralentizan su ejecución	Mayor rapidez al ejecutarse toda la lógica en el propio dispositivo
Coste	Los clientes no requieren grandes recursos	Los clientes deben estar actualizados, aumentando el coste
Centralización	Solo es necesario actualizar el servidor	Es necesario actualizar los dispositivos de los usuarios
Seguridad	Los mecanismos de seguridad se centran en el servidor	Se deben establecer mecanismos de seguridad de manera correcta en todos los clientes

Tabla 1. Diferencias entre aplicaciones web y aplicaciones de escritorio.



/ 3. Arquitectura cliente-servidor

Esta arquitectura hace uso de una distribución de las tareas entre dos lados: **el cliente y el servidor**. La conexión entre ambos lados se hace a través de una red, principalmente Internet, a través de la cual el cliente realiza las peticiones que el servidor procesa. Los resultados que se generan en el servidor son enviados al cliente también a través de la red que los conecta.

Dentro de esta arquitectura, el lado del servidor puede estar compuesto por más de una máquina, y por supuesto, por más de un programa. Habitualmente, el servidor se configura según una disposición multicapa o por niveles, compuesta de diferentes programas aumentando así la distribución del sistema. Habitualmente, son tres las capas o niveles que componen el servidor: **El servidor web, el servidor de base de datos y el intérprete o compilador**.

3.1. Proceso de comunicación

- El servidor iniciado está a la espera de peticiones del cliente.
- El cliente inicia una petición.
- El cliente espera la respuesta del servidor.
- El servidor procesa la petición.
- El servidor genera unas respuestas y la envía al cliente.
- El cliente recibe la respuesta.

3.2. Características

- Un cliente puede conectarse a varios servidores.
- Los usuarios interactúan con el cliente a través de una interfaz de usuario gráfica.
- La velocidad de conexión condiciona la comunicación.
- Un servidor puede aceptar peticiones de diferentes clientes.
- Un servidor no suele interactuar directamente con un usuario final.



Fig.2. Ejemplo de arquitectura cliente – servidor con varios clientes conectados a un servidor.

3.3. Ventajas e inconvenientes

Las ventajas y desventajas de esta arquitectura se muestran en la siguiente tabla:

VENTAJAS	INCONVENIENTES
Control centralizado	Velocidad
Escalabilidad	Incompatibilidades entre cliente y servidor
Facilidad de mantenimiento	Explotar recursos

Tabla 2. Ventajas e inconvenientes.



/ 4. Protocolos de comunicación

Para que la comunicación pueda desarrollarse de forma correcta entre el cliente y el servidor, es necesario establecer unos protocolos, es decir, unas reglas para que cliente y servidor puedan entenderse y que la comunicación sea efectiva. A continuación, describimos dos de los protocolos de arquitectura cliente-servidor más extendidos actualmente

4.1. Protocolo HTTP

HyperText Transfer Protocol, es el protocolo más utilizado en Internet desde 1990. La versión 1.0, que es la más utilizada, permite la transferencia de mensajes con encabezados que describen el contenido de los mensajes mediante la codificación MIME.

Permite la transferencia de archivos (principalmente en formato HTML, entre un navegador (cliente) y un servidor web (servidor), localizado mediante una cadena de caracteres denominada URL. Los pasos que sigue este protocolo son:

- El navegador realiza una solicitud HTTP.
- El servidor procesa la solicitud y envía una respuesta HTTP.

4.2. Protocolo HTTPS

Es la versión segura del protocolo anterior. HTTPS permite realizar la codificación de la comunicación mediante un certificado digital. Así se obtienen ciertas garantías de que la información que se envía no es interceptada y utilizada por terceros.

Cuando se accede a una página mediante este protocolo, en el navegador podemos ver que el prefijo de la URL **http://** se sustituye por **https://**. Además, en ciertos navegadores, se puede apreciar un icono y un color diferente, que indica que se está empleando un protocolo seguro.



Fig.3. Los certificados SSL se encargan de cifrar la información mediante el protocolo HTTPS.



Audio 1. "Certificados SSL"
<http://bit.ly/3HINhwu>



/ 5. Caso práctico 1: “Selección de arquitecturas”

Planteamiento: Pedro es el dueño de una clínica dental y quiere proporcionar a sus clientes un sistema para pedir cita a través de sus dispositivos móviles. Pedro está barajando las diferentes opciones que tiene y no sabe bien si crear *apps* nativas para las diferentes plataformas móviles o una aplicación web.

Nudo: Teniendo en cuenta las necesidades de Pedro y sabiendo que sus clientes utilizarán dispositivos muy heterogéneos, ¿cuál crees que sería la mejor opción para que Pedro diseñe su sistema de reserva de citas mediante dispositivos móviles: una aplicación nativa o una aplicación web?

Desenlace: Como hemos visto en el apartado de aplicaciones web y aplicaciones de escritorio, la mayoría de las empresas utiliza versiones tanto de escritorio como web para ofrecer sus servicios, de esa forma, deja en manos de sus usuarios, y en función de sus recursos, la elección de la opción más adecuada. Esta sería, por tanto, la opción más acertada para Pedro. Sin embargo, habitualmente podemos encontrarnos con que tenemos una limitación de recursos, en cuyo caso, la opción de una aplicación web sería la propicia por ser la que requiere un menor coste tanto para Pedro como para sus clientes.

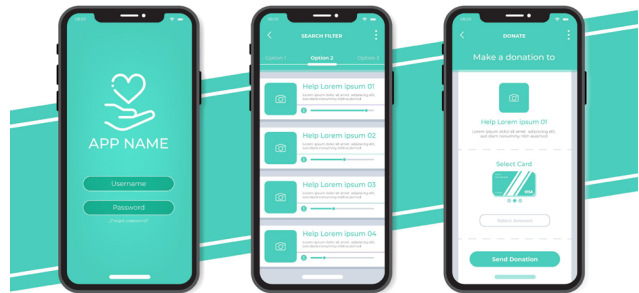


Fig.4. Plantilla de una aplicación móvil.

/ 6. Tecnologías en el lado del cliente

Teniendo en cuenta que la **arquitectura cliente-servidor** descarga la mayor parte de la ejecución en el servidor, la tecnología necesaria en el lado del cliente va a ser poca y sencilla.

6.1. Sistema operativo cliente

Se trata del software que gestiona los recursos del dispositivo y provee servicios a los programas que se ejecutan. Los principales Sistemas Operativos para clientes que existen son:

1. Para PC:

- » Windows.
- » Mac OS.
- » Unix.
- » Linux: Distribuciones más importantes (Ubuntu, Red Hat, Gentoo, OpenSUSE, Fedora).

2. Para móviles:

- » Android.
- » iOS.
- » BlackBerry OS.
- » Windows.



3. Para relojes:

- » Android.
- » WatchOS.

6.2. Navegador web

El *software* necesario para ejecutar una aplicación web en un cliente se conoce en la arquitectura cliente servidor como “cliente ligero”. Dentro del paradigma de las aplicaciones web, este programa recibe el nombre de navegador web.

Este *software* permite el acceso a páginas web, interpretando la información de diferentes tipos de archivos y sitios web para poder ser visualizados de manera apropiada.

Los principales navegadores web que existen en el mercado son:

- Google Chrome.
- Mozilla Firefox.
- Safari.
- Microsoft Edge.
- Internet Explorer.
- Opera.



Fig.5. Navegadores Web.

/ 7. Tecnologías en el lado del servidor

Como se comentaba en puntos anteriores, el servidor no estará compuesto por un solo equipo y un solo programa, habitualmente, se componen de diferentes programas que le dan una estructura multicapa, facilitando así su distribución.

7.1. Sistema operativo servidor

Se trata del software que **gestiona los recursos del dispositivo y provee servicios a los programas** que se ejecutan. Dentro del paradigma cliente servidor, al ser el servidor el dispositivo sobre el que se descarga la mayor parte del procesamiento y que dispone de los principales recursos, el sistema operativo en el servidor deberá tener mayor potencia y una gestión diferente para optimizar su eficiencia que el que haya en el lado del cliente. Los principales sistemas operativos de servidores son los siguientes:

1. **Linux** (por su potencia, seguridad, adaptación, y coste, es el más utilizado en todo el mundo). Las principales distribuciones son Ubuntu, Fedora, Debian y SuSE con sus versiones de servidor.
2. **Windows Server**.



3. Apple OS X Server.



Investigamos...

Gracias al software de virtualización podemos desvincular el sistema operativo del hardware subyacente. Esto nos permite tener diferentes sistemas operativos a través de máquinas virtuales. VirtualBox es un ejemplo de herramienta de virtualización gratuita y potente.

7.2. Servidor Web

Se trata de un *software* que procesa las peticiones del cliente y que genera respuestas en forma de código en un lenguaje web, que es interpretado por el cliente para que el usuario vea de manera apropiada una página web.



Fig.6. Logo del Servidor Apache.

El principal servidor web más utilizado en el mundo es el **servidor HTTP Apache**. Se trata de un programa de código abierto desarrollado para múltiples plataformas por la Apache Software Foundation que implementa el protocolo HTTP. Procesa las peticiones que recibe de parte de los diferentes clientes y genera el código de las páginas web como resultado de las peticiones para enviarlas a los clientes que las interpretan y muestran a los usuarios.

7.3. Servidor de bases de datos

Una vez que tengamos el sistema operativo y el servidor web instalado, debemos instalar y configurar un conjunto de componentes y módulos necesarios para dar soporte a las aplicaciones web: el servidor de bases de datos y el traductor del lenguaje de servidor.

Forma parte del sistema de gestión de base de datos, y permite el **almacenamiento, modificación y extracción de la información** de la base de datos que da soporte a la aplicación web. Algunos de los servidores de bases de datos más utilizados en el mundo son:

- MySQL.
- PostgreSQL.
- Microsoft SQL.
- MongoDB.



Fig.7. Logo del Servidor MySQL.

La base de datos de código abierto más popular del mundo es MySQL, siendo, además, idónea para la tarea de servidor de base de datos en una aplicación web.



7.4. Traductor del lenguaje (lógica del programa)

Por último, el servidor necesita tener un *software* de traducción, que permita dar la **lógica a nuestras aplicaciones** que estarán escritas en algún lenguaje de programación. Este *software* será un intérprete o un compilador. La inmensa mayoría de las aplicaciones web hacen uso de intérpretes.

Uno de los lenguajes más utilizados en aplicaciones web es PHP. Para que se puedan ejecutar de manera apropiada las aplicaciones será necesario tener instalado en el servidor el intérprete de PHP.



Fig.8. Lenguaje PHP.

/ 8. Infraestructura de tecnologías de servidor XAMP

Existen herramientas que integran las diferentes tecnologías que hemos descrito hasta ahora para montar de manera sencilla un servidor de aplicaciones web.

XAMP es un acrónimo de:

- X Para cualquier sistema operativo.
- A de Apache.
- M de MySQL.
- P de PHP.



Fig.9. Logo de infraestructura LAMP.

Por lo que una infraestructura XAMP se compone de un sistema operativo cualquiera (Generalmente **Linux**, **OS X** o **Windows** en sus versiones de servidor preferiblemente), **Apache Server** como servidor web, **MySQL** como SGBD y servidor de base de datos y **PHP** como intérprete del lenguaje de programación. Las versiones más utilizadas son **LAMP** (en entornos Linux), **WAMP** (en entornos Windows) y **MAMP** (en entornos Mac).

La instalación y configuración suele ser muy sencilla, tan solo deberemos descargar un paquete que suele tener una instalación y configuración que permite en pocos minutos tener instalado y en funcionamiento un servidor de aplicaciones web.

Enlaces de interés...

Para profundizar en el tema, te recomendamos que visites el siguiente enlace, donde podrás descargar un entorno XAMP para instalar en cualquier equipo: <https://www.apachefriends.org/es/>

En el siguiente vídeo podemos ver los pasos que hay que seguir para instalar una herramienta de este tipo y poder tener funcionando en cualquier equipo un servidor de aplicaciones web en unos pocos minutos.



Vídeo 1. "Instalación de la herramienta XAMPP"

<https://bit.ly/3JUTCrg>



/ 9. Conexión con el servidor remoto

Como desarrolladores de aplicaciones web, podemos optar por instalar nuestro servidor en una máquina a la que tengamos acceso físico, sin embargo, esto no es lo habitual. Normalmente, el servidor se encontrará en una ubicación remota y para conectarnos a él debemos utilizar determinadas herramientas.

En concreto, para poder subir los archivos de nuestras aplicaciones web al servidor, utilizaremos una herramienta de transferencia de archivos.

9.1. Protocolo FTP

Este protocolo establece unas normas para la transmisión de archivos entre diferentes equipos.

Este protocolo fija las reglas para que los ficheros sean enviados a través de una red TCP/IP. El protocolo es multiplataforma por lo que permite la transferencia entre equipos y sistemas diferentes.

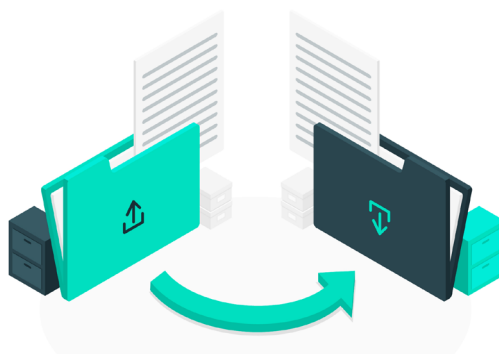


Fig.10. Transferencia de archivos.

9.2. FTP Seguro: FTPS

El protocolo FTPS añade una capa de seguridad al protocolo FTP. Mediante un ID de usuario o una contraseña, un certificado, o ambos. Mediante este protocolo podremos conectarnos a un servidor FTPS, comprobando primero si el certificado del servidor es confiable.

9.3. SFTP

Si bien FTPS añade una capa de seguridad a FTP, SFTP es un protocolo totalmente diferente, basado en el protocolo SSH. Este protocolo asegura la información encriptando y asegurando que no es accedida por terceros no autorizados.

9.4. Herramientas de transferencia

En el siguiente enlace de interés puedes encontrar las 9 herramientas FTP más utilizadas. En la actualidad, Filezilla es la herramienta más utilizada mundialmente por su versatilidad y por ser multiplataforma, gratuita y de código abierto.



Enlaces de interés...

Te recomendamos que visites la siguiente página web, donde podrás ver las nueve herramientas más utilizadas como clientes de transferencia de archivos: <https://www.ionos.es/digitalguide/hosting/cuestiones-tecnicas/los-mejores-clientes-ftp-para-el-alojamiento-web/>



/ 10. Caso práctico 2: “Instalamos XAMP”

Planteamiento: En el caso práctico anterior conocíamos a Pedro, que quería crear una aplicación para que sus clientes puedan pedir citas en su clínica a través de sus teléfonos móviles.

Nudo: Analiza qué tecnologías necesitamos para empezar a construir el prototipo de la aplicación que nos ha encargado Pedro. De entre las diferentes opciones que tenemos, ¿qué ventajas e inconvenientes presenta cada una?

Desenlace: Para el desarrollo del proyecto podemos optar por diferentes opciones.

1. **Instalar los sistemas y componentes necesarios para nuestro servidor de manera independiente.** Para ello, debemos instalar un sistema operativo, un servidor de aplicaciones web, un servidor de bases de datos y un intérprete de lenguaje PHP. Esta opción es más costosa en tiempo, ya que requiere realizar varias descargas, varias instalaciones y varias configuraciones. Por el contrario, nos deja más libertad a la hora de configurar según nuestras necesidades y recursos de una manera más personalizada.



Fig.11. Desarrollando una infraestructura con bases de datos.

2. **Instalar una infraestructura integrada XAMP** que incluye todos los componentes necesarios para tener funcionando el servidor de aplicaciones web. Esta opción es más rápida y sencilla, ya que requiere una única instalación y configuración, sin embargo, nos limita la flexibilidad y la adaptabilidad a nuestras necesidades.

/ 11. Resumen y resolución del caso práctico de la unidad

En este tema hemos estudiado la arquitectura y las tecnologías necesarias para que las aplicaciones web puedan funcionar. La **arquitectura cliente-servidor** divide en dos partes el funcionamiento de estas aplicaciones.

En el **servidor se ejecuta la lógica de la aplicación**, mediante un servidor web, un servidor de bases de datos y un intérprete de lenguaje. En el **cliente se reciben los resultados** de la ejecución a través de un cliente ligero denominado navegador web. Para comunicarse entre sí, cliente y servidor utilizan un conjunto de protocolos como HTTP, FTP y sus versiones seguras.

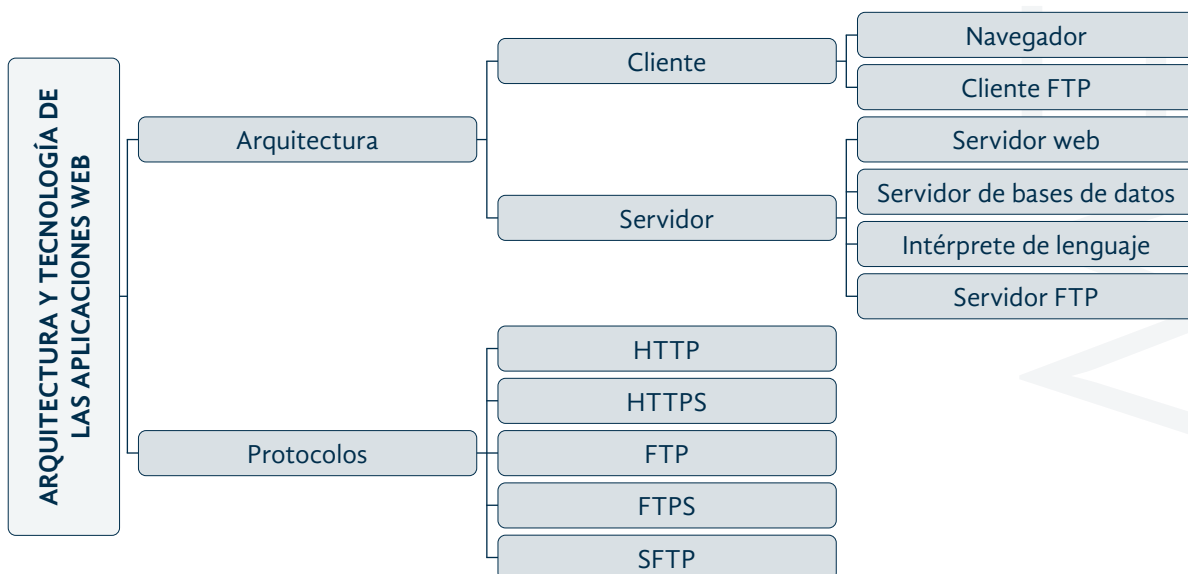


Fig.12. Esquema resumen del tema



Resolución del caso práctico de la unidad

Para la construcción de la aplicación web de la empresa de seguros será necesario implantar el siguiente *software*:

1. En el servidor

- » Sistema Operativo: Ubuntu Server.
- » Servidor Web: Apache Server.
- » Servidor de Bases de Datos: MySQL.
- » Intérprete de Lenguaje: PHP.
- » Servidor FTP: Filezilla Server.

2. En el cliente

- » Navegador web: Mozilla Firefox.
- » Cliente FTP: Filezilla Client.

Ten en cuenta que, al elegir tecnologías concretas, puede haber diferentes soluciones, por ejemplo, podríamos elegir como servidor de bases de datos uno diferente, como Microsoft SQL.

/ 12. Bibliografía

Martínez, J. F. (2014). *Implantación de aplicaciones web (GRADO SUP.)*. Grupo Editorial RA-MA.

Martínez, J. F. (2015). *Implantación de aplicaciones web en entornos internet, intranet y extranet*. Grupo Editorial RA-MA.

Cabello, A. L. C. (2015). *Implantación de aplicaciones web en entornos internet, intranet y extranet*. IFCD0210. Ic Editorial.

TALLEDO SAN MIGUEL, J. V. (2015). *Implantación de aplicaciones web en entornos internet, intranet y extranet*. Ediciones Paraninfo, S. A.